



DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

Project Report

Imitation of Television Remote Control System using STM32 MCU

Project Report

ARM BASED EMBEDDED SYSTEM DESIGN

Course Code: 2307220L

Second Year B.Tech. (Term-II)

2023 (NEP) Pattern



DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

Project Report

Imitation of Automated Lift Control System Using STM32 MCU

Project Report

ARM BASED EMBEDDED SYSTEM DESIGN

Course Code: 2307220L

GROUP NO: 3

Sr. No	Name	PRN	Division	Batch
1.	Shravani Lad	202401070151	B	B3
2.	Triveni Biradar	202401070172	B	B4
3.	Pratik Kadu	202401070198	C	C1

Second Year B.Tech. (Term-II)

2023 (NEP) Pattern

COURSE NAME: ARM Based Embedded System Design

COURSE CODE: 2307220L

COURSEOBJECTIVES:

2307220L.CEO.1: To provide students thorough understanding of the ARM development tools. 2307220L.CEO.2: To develop practical understanding of ARM architecture through experimentation. 2307220L.CEO.3: To integrate hardware and software in ARM-based systems.

COURSEOUTCOMES: After successful completion of the course, students will be able to,

2307220L.CO.1: Identify and describe functions of ARM-based development tools[L2].

2307220L.CO.2: Demonstrate the ability to write and debug programs for ARM

microcontrollers[L3]. 2307220L.CO.3: Utilize systematic problem-solving techniques for embedded systems[L4].

Project Problem Statement:-

Design and implement an Imitation of Television Remote Control System using STM32 MCU that allows users to change channels, adjust volume, mute audio, and power the TV ON/OFF using push-buttons. The system should display the selected channel on a 7-segment display and provide detailed status updates on an LCD screen. A relay will simulate TV power control, while a buzzer will generate alerts for user actions and power status changes.

Objectives:-

- To design a television remote control simulation using STM32 MCU.
- To control channel selection and volume adjustment using push-buttons.
- To display channel number and TV status using LCD and 7-segment display.
- To implement mute and power control functionality.
- To demonstrate real-time embedded system operation.

Learning Outcome:-

- Understanding of STM32-based embedded system design.
- Ability to interface LCD, 7-segment display, relay, and buzzer.
- Knowledge of implementing real-time control logic.
- Practical experience in hardware and software integration.

Relevance to Real-World Problems:-

Television remote systems are widely used in home entertainment devices. This project demonstrates how embedded systems can be used for device control, user interaction, and automation. It provides a practical understanding of remote control operations and display interfacing techniques.

Impact of Solving this Problem:-

- Improves understanding of embedded control systems.
- Demonstrates efficient user-interface management.
- Enhances knowledge of real-time display and control logic.
- Provides a foundation for consumer electronics automation systems.

Assumptions Made To Solve this Problem:-

- The STM32 MCU operates with stable power supply.
- Push-buttons function correctly without hardware faults.
- Relay simulates TV power instead of controlling an actual television.
- LCD and 7-segment displays are properly interfaced.
- Only one television system is simulated.

List of Components Used:-

1. STM32 Nucleo Development Board
2. Push Buttons
3. 2-Digit 7-Segment Display (Common Cathode)
4. 16x2 LCD Display
5. Relay Module
6. BC547 Transistor
7. 1N4148 / 1N4007 Diode
8. Buzzer
9. 220 Ω Resistor Network
10. Connecting Wires
11. USB Power Supply

Pin Configuration:-

1.)Switches (Push Buttons):-

Function	Pin
Power ON/OFF	PA0
Volume Up	PA1
Volume Down	PA4
Channel Up	PB12
Channel Down	PB13
Mute	PB14

Table 1:- Push buttons

1.) Output Devices:-

Function	Pin
Buzzer	PB5
Relay	PA5

Table 2:- Output Connections**2.) 7-Segment Display(Common Cathode):-**

Segment	Pin
A	PA8
B	PA9
C	PA10
D	PA11
E	PA12
F	PA13
G	PA14
Digit 1 (Tens)	PB3
Digit 2 (Ones)	PB4

Table 3:- 7 segment display**3.) LCD Display:-**

Signal Pin	Signal Pin
RS	PB6
EN	PB8
D4	PB9
D5	PB10
D6	PB1
D7	PB0

Table 4:- LCD display

Configurations Done in STM32CubeMX:-

- GPIO configured as input for push-buttons (Power, Channel, Volume, and Mute).
- GPIO configured as output for LCD display interfacing.
- GPIO configured as output for 7-segment display control.
- GPIO configured as output for relay and buzzer operation.
- Internal pull-up resistors enabled for active LOW buttons.
- System clock configured for STM32F410RBTx microcontroller operation.
- LCD interfacing configured in 4-bit communication mode.
- Real-time control logic implemented using HAL library functions.
- Main loop programmed to continuously monitor button inputs and update outputs.
- GPIO pins configured for relay driver and buzzer feedback operation.

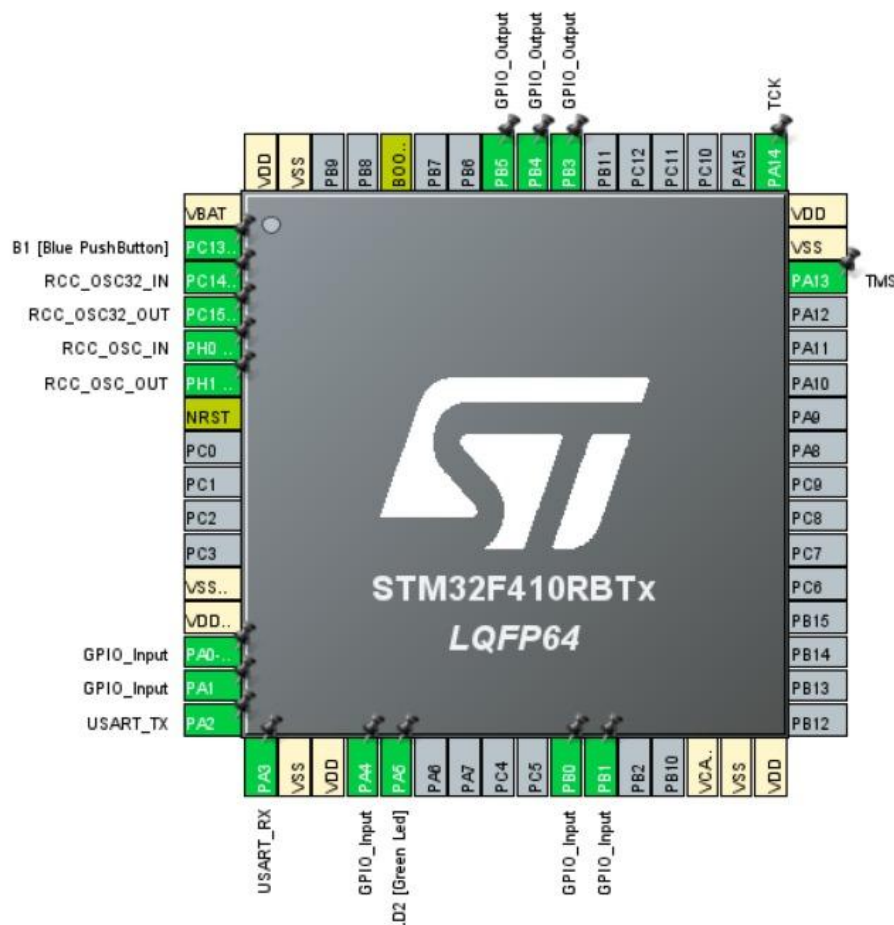


Fig 1:- Pinout Configuration

Code of STM32 CubeIDE:-

Main.c

```

/* USER CODE BEGIN Header */
/**
 * ****
 * @file      : main.c
 * @brief     : TV Remote Control System
 *           - Common Cathode 7-Segment (switch-case, active HIGH)
 *           - lcd16x2 library for LCD 16x2
 * ****
 */
/* USER CODE END Header */

/* Includes ----- */
#include "main.h"
#include "lcd16x2.h" // lcd16x2 library
#include <stdio.h>
#include <string.h>

/* Private typedef ----- */
typedef enum {
    TV_OFF = 0,
    TV_ON = 1
} TV_PowerState;

typedef struct {
    uint8_t channel;
    uint8_t volume;
    uint8_t mute;
    TV_PowerState powerState;
} TV_System;

/* Private define ----- */
// ===== PIN DEFINITIONS =====
// ----- Port A -----
#define POWER_BTN_PIN    GPIO_PIN_0 // PA0
#define VOL_UP_BTN_PIN   GPIO_PIN_1 // PA1
#define VOL_DOWN_BTN_PIN GPIO_PIN_4 // PA4
#define RELAY_PIN        GPIO_PIN_5 // PA5

// 7-Segment Segments (Port A) — Common Cathode: SET = ON, RESET = OFF
#define SEG_A_PIN        GPIO_PIN_8 // PA8 → segment A (top)
#define SEG_B_PIN        GPIO_PIN_9 // PA9 → segment B (top-right)
#define SEG_C_PIN        GPIO_PIN_10 // PA10 → segment C (bottom-right)
#define SEG_D_PIN        GPIO_PIN_11 // PA11 → segment D (bottom)
#define SEG_E_PIN        GPIO_PIN_12 // PA12 → segment E (bottom-left)
#define SEG_F_PIN        GPIO_PIN_13 // PA13 → segment F (top-left) [SWDIO reconfigured]

```



```

#define SEG_G_PIN      GPIO_PIN_14 // PA14 → segment G (middle) [SWDCK reconfigured]
#define SEG_DP_PIN     GPIO_PIN_15 // PA15 → decimal point (unused)

// ----- Port B -----
#define LCD_D7_PIN     GPIO_PIN_0  // PB0
#define LCD_D6_PIN     GPIO_PIN_1  // PB1
#define SEG_DIGIT1_PIN GPIO_PIN_3   // PB3 → Tens digit select (active HIGH)
#define SEG_DIGIT2_PIN GPIO_PIN_4   // PB4 → Ones digit select (active HIGH)
#define BUZZER_PIN     GPIO_PIN_5   // PB5
#define LCD_RS_PIN     GPIO_PIN_6   // PB6
#define LCD_RW_PIN     GPIO_PIN_7   // PB7
#define LCD_EN_PIN     GPIO_PIN_8   // PB8
#define LCD_D4_PIN     GPIO_PIN_9   // PB9
#define LCD_D5_PIN     GPIO_PIN_10  // PB10
#define CH_UP_BTN_PIN  GPIO_PIN_12  // PB12
#define CH_DOWN_BTN_PIN GPIO_PIN_13 // PB13
#define MUTE_BTN_PIN   GPIO_PIN_14  // PB14

// System parameters
#define DEBOUNCE_DELAY 250
#define BEEP_DURATION 50
#define BEEP_LONG_DURATION 200
#define MAX_CHANNEL 99
#define MIN_CHANNEL 1
#define MAX_VOLUME 20
#define MIN_VOLUME 0

/* Private variables -----*/
UART_HandleTypeDef huart2;

TV_System tv = {
    .channel = 1,
    .volume = 10,
    .mute = 0,
    .powerState = TV_OFF
};

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
void Delay_ms(uint32_t delay);
void BuzzerBeep(void);
void BuzzerBeepLong(void);
void ControlRelay(uint8_t state);
void Seg7_WriteDigit(uint8_t digit, uint8_t position);
void Seg7_WriteBlank(uint8_t position);
void Seg7_Update(void);
void LCD_ShowIdle(void);
void LCD_ShowPowerOn(void);
void LCD_ShowPowerOff(void);
void LCD_ShowChannelChange(void);
void LCD_ShowVolumeChange(void);
void LCD_ShowMuteStatus(void);
void LCD_ShowError(void);
void LCD_ShowStatus(void);

```

```

void ProcessPowerButton(void);
void ProcessChannelUp(void);
void ProcessChannelDown(void);
void ProcessVolumeUp(void);
void ProcessVolumeDown(void);
void ProcessMuteButton(void);

```

```

/* =====
Utility
===== */

```

```

void Delay_ms(uint32_t delay) { HAL_Delay(delay); }

```

```

void BuzzerBeep(void) {
    HAL_GPIO_WritePin(GPIOB, BUZZER_PIN, GPIO_PIN_SET);
    Delay_ms(BEEP_DURATION);
    HAL_GPIO_WritePin(GPIOB, BUZZER_PIN, GPIO_PIN_RESET);
}

```

```

void BuzzerBeepLong(void) {
    HAL_GPIO_WritePin(GPIOB, BUZZER_PIN, GPIO_PIN_SET);
    Delay_ms(BEEP_LONG_DURATION);
    HAL_GPIO_WritePin(GPIOB, BUZZER_PIN, GPIO_PIN_RESET);
}

```

```

void ControlRelay(uint8_t state) {
    HAL_GPIO_WritePin(GPIOA, RELAY_PIN,
        state ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

```

```

/* =====
7-SEGMENT DISPLAY — COMMON CATHODE
Common Cathode: segment ON = GPIO_PIN_SET (HIGH)
                  segment OFF = GPIO_PIN_RESET (LOW)
Digit select : position ON = GPIO_PIN_SET (transistor drives cathode LOW)
Pin map: PA8=A PA9=B PA10=C PA11=D PA12=E PA13=F PA14=G
=====

```

Segment layout:

```

  A
  ---
F |   | B
-G-
E |   | C
  ---
  D

```

Digit truth table (Common Cathode — 1 = SET/ON):

Digit	A	B	C	D	E	F	G
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1

```

7 | 1 1 1 0 0 0 0
8 | 1 1 1 1 1 1 1
9 | 1 1 1 1 0 1 1

```

```

===== */
/* =====
7-SEGMENT DISPLAY — COMMON CATHODE (Active HIGH)
Segments: A(PA8), B(PA9), C(PA10), D(PA11), E(PA12), F(PA13), G(PA14)
Digit select: DIGIT1(PB3), DIGIT2(PB4) — active HIGH
===== */

void Seg7_WriteDigit(uint8_t digit, uint8_t position) {
    // First, clear all segment pins (turn OFF)
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 |
        GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14,
        GPIO_PIN_RESET);

    // Set segments according to digit (active HIGH)
    switch (digit) {
        case 0:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET); // D
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET); // E
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET); // F
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET); // G
            break;
        case 1:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
            break;
        case 2:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET); // D
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET); // E
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET); // G
            break;
        case 3:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET); // D
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET); // G
            break;
        case 4:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET); // F
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET); // G
            break;
        case 5:
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET); // D

```

```

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET); // F
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET); // G
    break;
case 6:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET); // D
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET); // E
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET); // F
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET); // G
    break;
case 7:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
    break;
case 8:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET); // D
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET); // E
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET); // F
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET); // G
    break;
case 9:
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET); // A
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); // B
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // C
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET); // D
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET); // F
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET); // G
    break;
default:
    // blank (all segments already cleared)
    break;
}

// Digit select (active HIGH) — only one digit at a time
if (position == 0) { // tens digit
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);
} else { // ones digit
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET);
}
}

void Seg7_WriteBlank(uint8_t position) {
    // Turn off all segments
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 |
        GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14,
        GPIO_PIN_RESET);
    // Select the digit to blank (keeps multiplexing consistent)
    if (position == 0) {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_SET);
    }
}

```

```

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);
} else {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET);
}
}

```

```

void Seg7_Update(void) {
    if (tv.powerState == TV_ON) {
        uint8_t tens = tv.channel / 10;
        uint8_t ones = tv.channel % 10;

        // Show tens digit (if zero, blank it)
        if (tens == 0) {
            Seg7_WriteBlank(0);
        } else {
            Seg7_WriteDigit(tens, 0);
        }
        HAL_Delay(5); // multiplexing delay

        // Always show ones digit
        Seg7_WriteDigit(ones, 1);
        HAL_Delay(5);
    } else {
        // TV OFF: turn off both digits and segments
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 |
            GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14,
            GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3 | GPIO_PIN_4, GPIO_PIN_RESET);
    }
}

```

```

/* =====
LCD DISPLAY — lcd16x2 library
Functions match the sample format provided:
    lcd16x2_clear()
    lcd16x2_setCursor(row, col)
    lcd16x2_printf(fmt, ...)
===== */

```

```

void LCD_ShowIdle(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf("TV is OFF");
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf("Press POWER ON");
}

```

```

void LCD_ShowPowerOn(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf("TV POWER ON");
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf("CH:%02d VOL:%02d", tv.channel, tv.volume);
}

```

```

void LCD_ShowPowerOff(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf("TV POWER OFF");
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf("Goodbye!");
}

void LCD_ShowChannelChange(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf("Channel: %02d", tv.channel);
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf("Vol:%02d", tv.volume);
}

void LCD_ShowVolumeChange(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf("Volume: %02d", tv.volume);
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf(tv.mute ? "MUTE ON" : "MUTE OFF");
}

void LCD_ShowMuteStatus(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf(tv.mute ? "MUTE ACTIVATED" : "MUTE DEACTIVATED");
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf("Volume: %02d", tv.volume);
}

void LCD_ShowError(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf("ERROR!");
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf("TURN ON TV FIRST");
}

/* Normal running status shown after every action */
void LCD_ShowStatus(void) {
    lcd16x2_clear();
    lcd16x2_setCursor(0, 0);
    lcd16x2_printf("CH:%02d VOL:%02d", tv.channel, tv.volume);
    lcd16x2_setCursor(1, 0);
    lcd16x2_printf(tv.mute ? "MUTE ON" : "MUTE OFF");
}

/* =====
Button handlers
===== */

void ProcessPowerButton(void) {
    BuzzerBeepLong();
    if (tv.powerState == TV_OFF) {

```

```

        tv.powerState = TV_ON;
        ControlRelay(1);
        LCD_ShowPowerOn();
        Delay_ms(1000);
        LCD_ShowStatus();
    } else {
        tv.powerState = TV_OFF;
        ControlRelay(0);
        LCD_ShowPowerOff();
        Delay_ms(1000);
        LCD_ShowIdle();
    }
    Seg7_Update();
}

void ProcessChannelUp(void) {
    if (tv.powerState == TV_OFF) {
        LCD_ShowError();
        BuzzerBeep();
        Delay_ms(1000);
        LCD_ShowIdle();
        return;
    }
    BuzzerBeep();
    tv.channel = (tv.channel < MAX_CHANNEL) ? tv.channel + 1 : MIN_CHANNEL;
    LCD_ShowChannelChange();
    Delay_ms(500);
    LCD_ShowStatus();
    Seg7_Update();
}

void ProcessChannelDown(void) {
    if (tv.powerState == TV_OFF) {
        LCD_ShowError();
        BuzzerBeep();
        Delay_ms(1000);
        LCD_ShowIdle();
        return;
    }
    BuzzerBeep();
    tv.channel = (tv.channel > MIN_CHANNEL) ? tv.channel - 1 : MAX_CHANNEL;
    LCD_ShowChannelChange();
    Delay_ms(500);
    LCD_ShowStatus();
    Seg7_Update();
}

void ProcessVolumeUp(void) {
    if (tv.powerState == TV_OFF) {
        LCD_ShowError();
        BuzzerBeep();
        Delay_ms(1000);
        LCD_ShowIdle();
        return;
    }
    BuzzerBeep();

```

```

    if (tv.volume < MAX_VOLUME) {
        tv.volume++;
        if (tv.mute) tv.mute = 0;
    }
    LCD_ShowVolumeChange();
    Delay_ms(500);
    LCD_ShowStatus();
}

void ProcessVolumeDown(void) {
    if (tv.powerState == TV_OFF) {
        LCD_ShowError();
        BuzzerBeep();
        Delay_ms(1000);
        LCD_ShowIdle();
        return;
    }
    BuzzerBeep();
    if (tv.volume > MIN_VOLUME) {
        tv.volume--;
        if (tv.mute) tv.mute = 0;
    }
    LCD_ShowVolumeChange();
    Delay_ms(500);
    LCD_ShowStatus();
}

void ProcessMuteButton(void) {
    if (tv.powerState == TV_OFF) {
        LCD_ShowError();
        BuzzerBeep();
        Delay_ms(1000);
        LCD_ShowIdle();
        return;
    }
    BuzzerBeep();
    tv.mute = !tv.mute;
    LCD_ShowMuteStatus();
    Delay_ms(500);
    LCD_ShowStatus();
}

/* =====
Main
===== */

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();

    // Initialize LCD via lcd16x2 library
    // Pins: RS=PB6, RW=PB7, EN=PB8, D4=PB9, D5=PB10, D6=PB11, D7=PB0
    lcd16x2_init(GPIOB, LCD_RS_PIN, LCD_RW_PIN, LCD_EN_PIN,
        LCD_D4_PIN, LCD_D5_PIN, LCD_D6_PIN, LCD_D7_PIN);

```



```

// Startup splash
lcd16x2_clear();
lcd16x2_setCursor(0, 0);
lcd16x2_printf("TV Remote System");
lcd16x2_setCursor(1, 0);
lcd16x2_printf("Initializing...");
Delay_ms(2000);

ControlRelay(0);
LCD_ShowIdle();
Seg7_Update();

while (1) {
    if (HAL_GPIO_ReadPin(GPIOA, POWER_BTN_PIN) == GPIO_PIN_RESET) {
        ProcessPowerButton();
        Delay_ms(DEBOUNCE_DELAY);
    }
    else if (HAL_GPIO_ReadPin(GPIOB, CH_UP_BTN_PIN) == GPIO_PIN_RESET) {
        ProcessChannelUp();
        Delay_ms(DEBOUNCE_DELAY);
    }
    else if (HAL_GPIO_ReadPin(GPIOB, CH_DOWN_BTN_PIN) == GPIO_PIN_RESET) {
        ProcessChannelDown();
        Delay_ms(DEBOUNCE_DELAY);
    }
    else if (HAL_GPIO_ReadPin(GPIOA, VOL_UP_BTN_PIN) == GPIO_PIN_RESET) {
        ProcessVolumeUp();
        Delay_ms(DEBOUNCE_DELAY);
    }
    else if (HAL_GPIO_ReadPin(GPIOA, VOL_DOWN_BTN_PIN) == GPIO_PIN_RESET) {
        ProcessVolumeDown();
        Delay_ms(DEBOUNCE_DELAY);
    }
    else if (HAL_GPIO_ReadPin(GPIOB, MUTE_BTN_PIN) == GPIO_PIN_RESET) {
        ProcessMuteButton();
        Delay_ms(DEBOUNCE_DELAY);
    }
}

Seg7_Update(); // keep 7-seg multiplexed
Delay_ms(10);
}

/* =====
System Clock — HSI PLL → 84 MHz
===== */
void SystemClock_Config(void) {
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;

```

```

RCC_OscInitStruct.PLL.PLLM      = 16;
RCC_OscInitStruct.PLL.PLLN      = 336;
RCC_OscInitStruct.PLL.PLLP      = RCC_PLLP_DIV4;
RCC_OscInitStruct.PLL.PLLQ      = 4;
RCC_OscInitStruct.PLL.PLLR      = 2;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) Error_Handler();
RCC_ClkInitStruct.ClockType      = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCCLK
    | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource   = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK) Error_Handler();
}

```

```

/* =====
USART2
===== */

```

```

static void MX_USART2_UART_Init(void) {
    huart2.Instance      = USART2;
    huart2.Init.BaudRate  = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits  = UART_STOPBITS_1;
    huart2.Init.Parity     = UART_PARITY_NONE;
    huart2.Init.Mode       = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl  = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart2) != HAL_OK) Error_Handler();
}

```

```

/* =====
GPIO Init
===== */

```

```

static void MX_GPIO_Init(void) {
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    // ---- Button inputs — pull-up, active LOW ----
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    GPIO_InitStruct.Pin = POWER_BTN_PIN | VOL_UP_BTN_PIN | VOL_DOWN_BTN_PIN;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = CH_UP_BTN_PIN | CH_DOWN_BTN_PIN | MUTE_BTN_PIN;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    // ---- Outputs ----
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

```

```

// PA: Relay + 7-seg segments (PA13/PA14 reconfigured from SWD)

```

```

GPIO_InitStruct.Pin = RELAY_PIN |
                      SEG_A_PIN | SEG_B_PIN | SEG_C_PIN | SEG_D_PIN |
                      SEG_E_PIN | SEG_F_PIN | SEG_G_PIN | SEG_DP_PIN;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

// PB: LCD lines + buzzer + digit selects
GPIO_InitStruct.Pin = LCD_D7_PIN | LCD_D6_PIN |
                      SEG_DIGIT1_PIN | SEG_DIGIT2_PIN |
                      BUZZER_PIN |
                      LCD_RS_PIN | LCD_RW_PIN | LCD_EN_PIN |
                      LCD_D4_PIN | LCD_D5_PIN;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
}

void Error_Handler(void) {
    __disable_irq();
    while (1);
}

```

lcd16x2.c

```

/**
 * lcd16x2.c — HD44780 16x2 LCD driver (4-bit mode)
 *
 * All pins must be on the SAME GPIO port (passed to lcd16x2_init).
 * Pin wiring used in TV Remote project:
 * Port B: RS=PB6, RW=PB7, EN=PB8, D4=PB9, D5=PB10, D6=PB11, D7=PB12
 */

#include "lcd16x2.h"
#include <stdio.h>
#include <string.h>

/* ---- Internal state ---- */
static GPIO_TypeDef *_port;
static uint16_t _rs, _rw, _en;
static uint16_t _d4, _d5, _d6, _d7;

/* ---- LCD commands ---- */
#define CMD_CLEAR      0x01
#define CMD_HOME      0x02
#define CMD_ENTRY_MODE 0x06 /* increment, no shift */
#define CMD_DISPLAY_ON 0x0C /* display on, cursor off, blink off */
#define CMD_FUNCTION_SET 0x28 /* 4-bit, 2-line, 5x8 */
#define CMD_SET_DDRAM  0x80

/* =====
Low-level helpers
===== */

static inline void _delay(uint32_t ms) { HAL_Delay(ms); }

/* Pulse the EN pin to latch data */
static void _pulse_en(void) {

```

```

    HAL_GPIO_WritePin(_port, _en, GPIO_PIN_SET);
    _delay(1);
    HAL_GPIO_WritePin(_port, _en, GPIO_PIN_RESET);
    _delay(1);
}

/* Send one nibble (upper 4 bits of value) on D4-D7 */
static void _send_nibble(uint8_t nibble) {
    /* nibble bits 0-3 map to D4-D7 */
    HAL_GPIO_WritePin(_port, _d4, (nibble >> 0) & 0x01 ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(_port, _d5, (nibble >> 1) & 0x01 ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(_port, _d6, (nibble >> 2) & 0x01 ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(_port, _d7, (nibble >> 3) & 0x01 ? GPIO_PIN_SET : GPIO_PIN_RESET);
    _pulse_en();
}

/* Send a full byte in two nibbles: high nibble first, then low nibble */
static void _send_byte(uint8_t byte, uint8_t is_data) {
    /* RS: 1 = data, 0 = command */
    HAL_GPIO_WritePin(_port, _rs, is_data ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(_port, _rw, GPIO_PIN_RESET); /* always write */

    _send_nibble(byte >> 4); /* high nibble */
    _send_nibble(byte & 0x0F); /* low nibble */

    _delay(2);
}

/* =====
Public API
===== */

/**
 * lcd16x2_init — initialise the LCD in 4-bit mode.
 *
 * All pins MUST be on the same port.
 * Call once after MX_GPIO_Init().
 */
void lcd16x2_init(GPIO_TypeDef *port,
                  uint16_t rs_pin, uint16_t rw_pin, uint16_t en_pin,
                  uint16_t d4_pin, uint16_t d5_pin,
                  uint16_t d6_pin, uint16_t d7_pin)
{
    _port = port;
    _rs = rs_pin;
    _rw = rw_pin;
    _en = en_pin;
    _d4 = d4_pin;
    _d5 = d5_pin;
    _d6 = d6_pin;
    _d7 = d7_pin;

    /* HD44780 power-on sequence */
    _delay(50);

    HAL_GPIO_WritePin(_port, _rs | _rw, GPIO_PIN_RESET);

```

```

/* Step 1-3: send nibble 0x3 three times (8-bit reset) */
for (int i = 0; i < 3; i++) {
    _send_nibble(0x3);
    _delay(5);
}

/* Step 4: send nibble 0x2 → switches to 4-bit mode */
_send_nibble(0x2);
_delay(5);

/* Now use full 2-nibble commands */
_send_byte(CMD_FUNCTION_SET, 0); /* 4-bit, 2-line, 5x8 */
_send_byte(CMD_DISPLAY_ON, 0); /* display on */
_send_byte(CMD_CLEAR, 0); /* clear display */
_delay(2);
_send_byte(CMD_ENTRY_MODE, 0); /* entry mode */
}

/** Clear display and return cursor to home */
void lcd16x2_clear(void) {
    _send_byte(CMD_CLEAR, 0);
    _delay(2);
}

/** Move cursor to (0,0) without clearing */
void lcd16x2_home(void) {
    _send_byte(CMD_HOME, 0);
    _delay(2);
}

/**
 * lcd16x2_setCursor — move cursor to row/col.
 * row: 0 = first line, 1 = second line
 * col: 0–15
 */
void lcd16x2_setCursor(uint8_t row, uint8_t col) {
    uint8_t addr = (row == 0) ? (CMD_SET_DDGRAM + col)
        : (CMD_SET_DDGRAM + 0x40 + col);
    _send_byte(addr, 0);
}

/** Write a single character */
void lcd16x2_putChar(char c) {
    _send_byte((uint8_t)c, 1);
}

/** Write a null-terminated string */
void lcd16x2_print(const char *str) {
    while (*str) {
        lcd16x2_putChar(*str++);
    }
}

/**
 * lcd16x2_printf — printf-style formatted output to LCD.

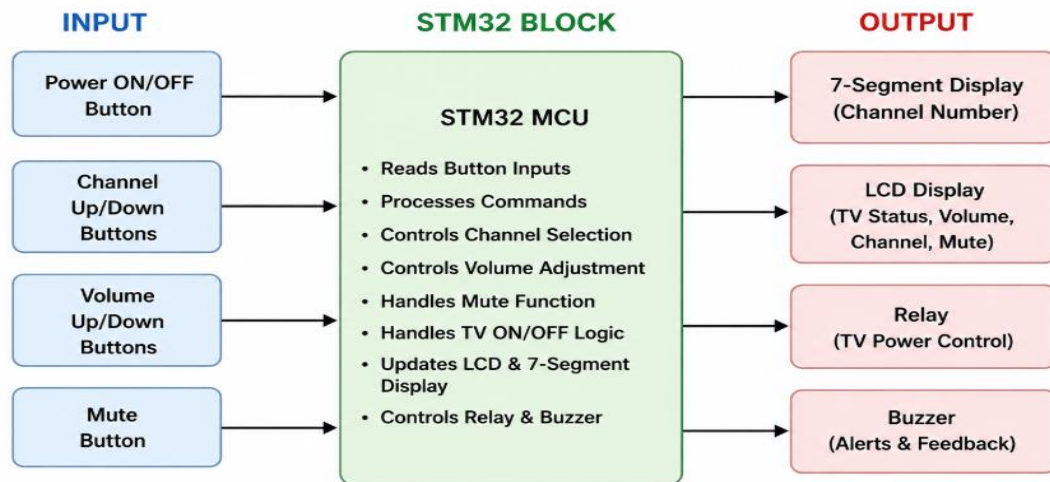
```

```
* Maximum formatted string length: 32 characters.  
*/  
void lcd16x2_printf(const char *fmt, ...) {  
    char buf[33];  
    va_list args;  
    va_start(args, fmt);  
    vsnprintf(buf, sizeof(buf), fmt, args);  
    va_end(args);  
    lcd16x2_print(buf);  
}
```

Block diagram:-

Fig 2:- System Block Diagram

TELEVISION REMOTE CONTROL SYSTEM



Result:-

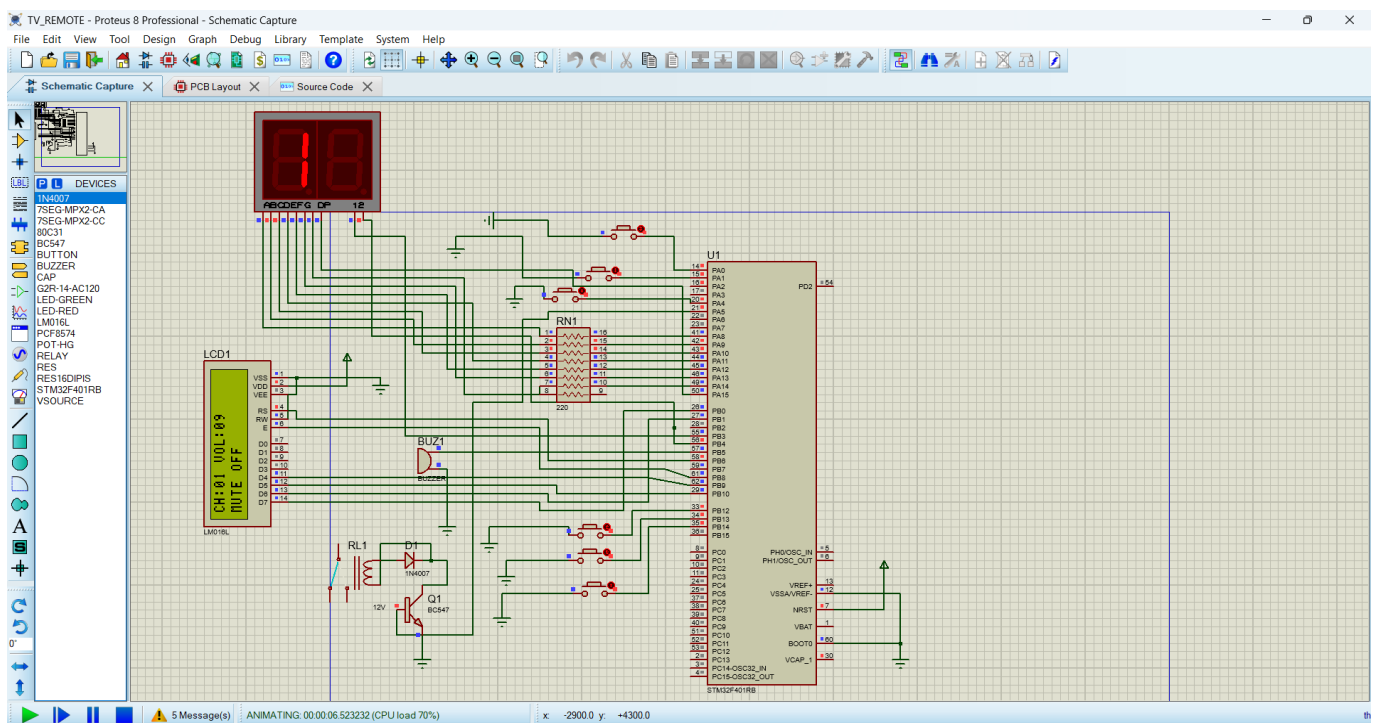


Fig 3:- Proteus Simulation

Challenges Faced:-

- Configuring GPIO pins for multiple peripherals.
- Synchronizing LCD and 7-segment display updates.
- Handling multiple button inputs efficiently.
- Interfacing relay driver circuit properly.
- Managing real-time state transitions.

Conclusion:-

The Imitation of Television Remote Control System using STM32 MCU was successfully designed and implemented using push-buttons, LCD display, 7-segment display, relay, and buzzer interfacing. The system effectively simulated basic television remote operations such as channel selection, volume adjustment, mute control, and TV power ON/OFF functionality. The 7-segment display dynamically showed the selected channel number, while the LCD displayed real-time system status and control information.

The relay module successfully simulated television power control, and the buzzer provided audio feedback for button presses and status changes. The project demonstrated efficient real-time embedded system operation through GPIO interfacing, display management, and state-based control logic using STM32 microcontroller programming.

Through this project, practical understanding of STM32CubeMX configuration, Embedded C programming, peripheral interfacing, and hardware-software integration was achieved. Overall, the project provided valuable knowledge of embedded automation systems and consumer electronics control applications, forming a strong foundation for advanced IoT and smart device projects.